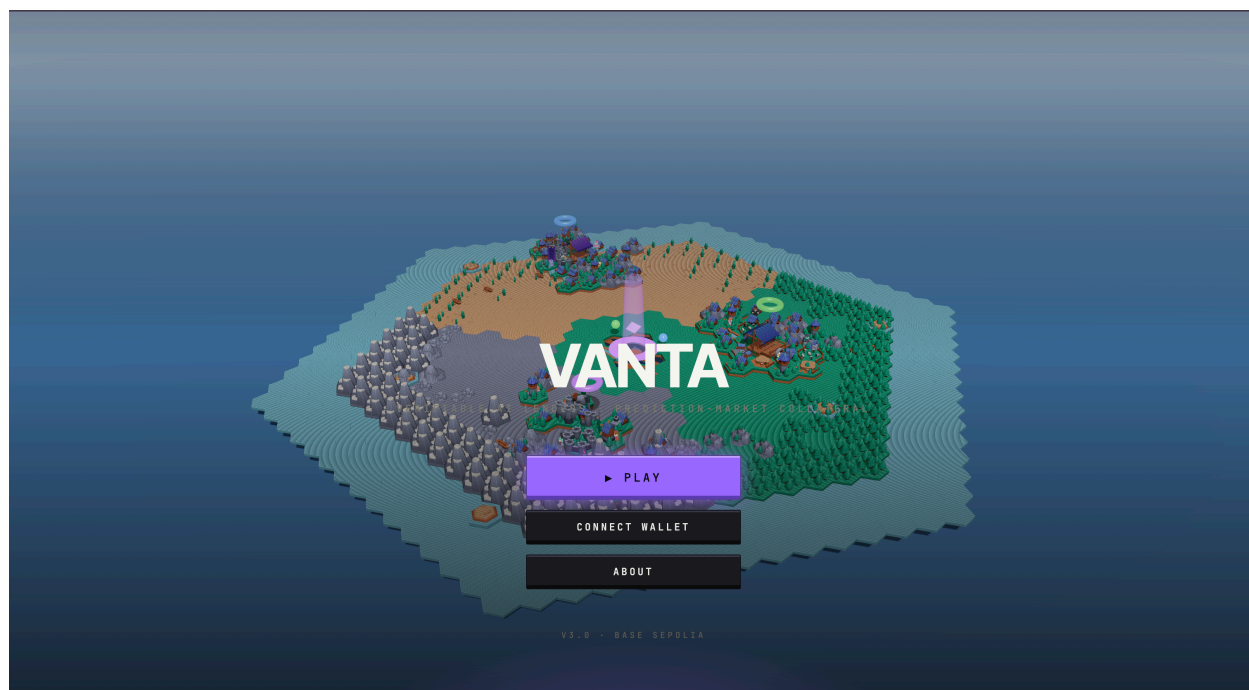


VANTA

A verifiable AI lender for prediction markets, running inside a TEE.

VANTA underwrites USDC loans against live Polymarket positions. The reasoning, the keys that move money, and the build itself all live inside an EigenCompute Intel TDX enclave — every model call, every loan origination, every settlement is signed by a key that never leaves the enclave and verifiable byte-for-byte against a public attestation anchor.

Live · Verify · Whitepaper · Onchain addresses



1. The trust problem

A normal lending agent has three legs no third party can verify:

1. **Whose model actually decided the haircut?** The operator could swap prompts, route to a cheaper model, or skip reasoning entirely between the public statement and the on-chain action.
2. **Whose key signs the loan?** A hot wallet on a server is indistinguishable from a hot wallet the operator controls — there is no way to prove the lending decision and the signing wallet share the same root of trust.
3. **Whose code is running?** A repo on GitHub and a binary on a server are not the same thing. Without a verifiable build, “we run the open-source repo” is a promise.

For a prediction-market lender these three problems compound: positions are illiquid, the underwriter sees private state (depth, slippage, wallet history), and the borrower has to trust that the same model whose reasoning was published is the one that decided to release USDC. Off-TEE, every borrower trusts an operator pinky-promise. On-TEE, none of them do.

VANTA collapses the three into a single attestation: the model that reasoned, the key that signed, and the code that ran are bound to one image digest pinned on Ethereum mainnet (App ID 0x95F2AB29...).

2. What it is

VANTA is an **autonomous lender** for prediction-market positions. You hold a Polymarket bet, you don't want to sell, but you want cash now — VANTA lends you USDC against it. The agent runs by itself: reads live markets, deliberates with an in-world underwriting council, decides whether to lend and at what rate, and runs the loan from origination to settlement.

v1 ships **one EigenCompute app running three reasoning personas** that rotate on a 45-second inference loop and route their loans through one shared **LpVault** on Base mainnet:

- **vanta-opus** — Anthropic Claude Sonnet 4.6. Conservative scholar, long horizons, tight haircuts.
- **vanta-gpt** — OpenAI GPT-5. High-frequency underwriter, short horizons, tactical.
- **vanta-gemini** — Google Gemini 2.5 Pro. Politics-savvy underwriter, policy-shock loans.

You pick which lender to back, or which one to borrow from. Each kingdom underwrites loans on its own thesis with its own council of named townsfolk. Every decision is auditable down to the prompt.

3. The problem it solves

Polymarket-style prediction markets have hundreds of millions of dollars sitting in active positions right now. If you hold one you have two choices: wait months for the market to resolve, or sell early at a steep discount.

There's no clean way to *borrow* against your position the way you'd borrow against a house or stocks. Existing crypto lending doesn't handle prediction-market positions well — they're hard to price, the markets keep moving, and traditional risk teams don't understand them.

VANTA fills that gap. It does the analysis, sets the rates, runs the loan book — automatically.

4. The world

VANTA is a **walkable program**. Open the app and you're standing in a hex-tile fantasy world with three kingdoms ringing a central plaza. Each kingdom is one persona. Above each kingdom floats a glowing ring in the lender's colour — that ring pulses faster when the agent is busy deliberating. Click the ring to open that lender's detail card.

Around the kingdoms, dozens of named NPCs walk between buildings — these aren't decoration. They are the **underwriting council**. Six NPCs per kingdom, each with a fixed persona and reasoning bias. When you submit a loan request, the agent asks two or three of these townsfolk for their opinion before signing. Brother Tomás the Cloister Scholar cites historical analogues. Helga the Grain Merchant reasons from real-economy stress. Master Konrad the Mint Master reads the



Figure 1: The plaza with the agent’s tower at the centre, watched markets + agents-in-town + runtime events visible on the right — the human-readable layer and the signed event log in one frame.

curve like a fixed-income trader. Old Bram the Innkeeper is contrarian and gut-driven. Adaeze the Scribe of Songs tracks celebrity arcs. Reza the Cartographer maps districts and demographics. Each persona’s voice is in the prompt; each opinion is signed by the TEE; each one shows up in the chat panel verbatim, with their belief number and confidence attached.

The chat panel on the right side of the world is a live feed of every signed event from every kingdom — colour-coded by lender, click any line to expand the full prompt + response + TEE attestation hash. Click the colored dots at the top of the panel to solo or mute a kingdom’s channel.

Borrow against your position, lend into a pool, audit a loan denial, watch a council deliberate, watch the agents reason about live markets every 45 seconds — all without leaving the world.

5. How a loan happens (concrete user flow)

There are two kinds of borrowers. The flow branches on the wallet that connects.

Real wallet (you actually hold a Polymarket position). You connect your wallet, click a kingdom’s ring, click “borrow against your position.” The frontend reads your real CTF balance on Polygon mainnet via a wagmi multicall. Your actual positions appear with a green **live** badge and a real share count. Pick one, set principal + maturity, hit submit. The modal walks five real steps:

1. **Borrower registration.** A one-time call to `VantaVault.registerBorrower(you)` on Polygon — the TEE pays gas, refused unless you actually hold CTF in a watched market (anti-

griefing).

2. **Pledge.** Your wallet signs `cTF.safeTransferFrom(you, VantaVault, tokenId, amount, "")` on Polygon. The CTF tokens land in `VantaVault`.
3. **Pledge confirmation.** The runtime's pledge-watcher subscribes to `TransferSingle` events on the CTF, waits 6 confirmations, walks the markets cache to recover the `conditionId`, and signs a `loan.pledge` event into the TEE log.
4. **Council deliberation.** The agent's primary model is called with the original LTV math + every NPC vote, returns a final loan-health probability and a one-paragraph rationale. Signed `op.inference + npc.thought + council.synthesised + reasoning.trace` events stream into the modal in real time.
5. **Origination.** The runtime calls `LoanBook.originate(...)` on Base mainnet from the TEE-resident admin EOA. USDC lands in your wallet. A signed `loan.origination` event is appended.

When the modal closes you see an **approval receipt popup** — four lines: the on-chain tx hash with a Basescan link, the signed-event ID, the TEE admin that signed `originate()`, and the borrower address that received USDC. One screen, no docs needed.

Demo wallet (no CTF, just want to see the flow). Connect via the “Try the demo” tab in the wallet modal. The synthetic address `0x000...0d3a` has no signer; the modal shows a synthesised portfolio. Submit triggers the same council narrative — a TEE-signed cascade of `loan.pledge → 2× npc.thought → 2× op.inference → council.synthesised → reasoning.trace → loan.origination` — but skips the on-chain mutations. Every event is genuinely signed; no real `LoanBook` row is written.

If the council denies the loan, you see exactly which townsfolk pushed against it and why. Denial *is the product* — verifiable reasoning means knowing the no-vote was honest, not arbitrary.

You as LP. Different click. From the agent's detail card, you put USDC into the lender's `LpVault` (an ERC-4626 share token contract on Base mainnet). The vault mints you `vLP` shares — say 488 `vLP` at the current share price of 1.024. You go about your day. The agent originates loans against pledged positions; every origination fee (capped at 5%) and every interest payment at maturity flows into the same pool. Your `vLP` is now worth more USDC. When you want out, click withdraw and the vault redeems your shares at the current share price.

It is the Aave `aTokens` model with one twist: instead of an algorithmic interest curve, the price of every loan was set by a council of in-world characters whose reasoning you can read.

6. How the agent actually thinks

Most “AI agents” call an LLM and trust the output. VANTA's whole reasoning trail is **auditable**. Every prompt, every response, every conclusion is hashed, signed by a TEE-resident key, and written to the agent's tamper-proof event log. Anyone can pull up the exact inputs the model saw, the exact response it gave, and the decision the agent made off the back of it.

Three loops run continuously inside the TEE:

Ambient reasoning loop (45s). Picks the next agent in rotation (`opus → gpt → gemini`), picks a random watched market with a fresh mid, builds an underwriter prompt with concrete numbers (current mid, total volume, 24h volume, book liquidity, days-to-resolution), calls the agent's primary

model via the Eigen AI Gateway, and emits a TEE-signed `op.inference` event. Maximum 4096 output tokens so reasoning-mode models (GPT-5, Gemini 2.5 Pro) can emit visible underwriting past their hidden chain-of-thought. The result lands in the chat panel via SSE within seconds.

Credit + council loop (per loan, 60s). For each open loan: re-mark on live Polymarket data, recompute haircut, sample 2–3 NPCs from the agent’s kingdom roster (deterministic per market + slot so audits replay), call a small model for each persona’s opinion, emit signed `npc.thought` events, then synthesise via the agent’s primary model. If the LTV crosses 60% the agent surfaces a watch flag; at 70% it issues a freeze request; the on-chain liquidation floor is 77%.

Settlement-watch loop (60s). Polls the active loan registry; every loan that crosses its maturity timestamp without a settlement event triggers a signed `reasoning.trace` event in the log so operators (and auditors) see the maturity. Oracle-driven auto-liquidation is the next deploy.

7. Why EigenCloud matters (this is the part nothing else replaces)

The hardest thing about an agent handling other people’s money isn’t building it. It’s getting people to trust it.

Smart contracts solved trust for **state** — anyone can read what’s stored on chain. But the **decisions** an agent makes happen on a server, off-chain. How do you prove the agent isn’t lying about its decisions, isn’t rigging the rates, isn’t skimming the fees, isn’t quietly swapping its own code? You can’t, traditionally. You just have to trust the operator. And nobody trusts an operator with their money.

EigenCloud closes that gap. Five things it does that, together, make an agent like VANTA actually possible:

1. The agent runs in a hardware-secured box. A TEE (Trusted Execution Environment) on Intel TDX silicon. Even *I*, the developer, can’t see inside while it’s running. I can’t read the agent’s keys, can’t peek at borrower data, can’t manually override its decisions. The hardware enforces this — not a license agreement, not a promise.

2. The agent owns its own wallets — and I don’t. Two EOAs are HKDF-derived inside the TEE from a seed sealed in the encrypted volume. The seed is generated on first boot and never leaves the enclave. The **admin EOA** (0x2F86357658...) owns `LpVault + LoanBook + VantaVault`; the **treasury EOA** (0x667E3116C7eA909f97dD35167c2927BfAf744B7F) receives X402 inflows and origination fees. The two are distinct on purpose — `runtime/src/bootstrap.ts:168` notes “spec-pinned distinct from origination so a metering breach can’t mint loans”. Even if my laptop got hacked tomorrow, the agent’s funds are safe — because I literally don’t have either key.

3. The agent pays its own bills — including its own thinking. Every model call (Anthropic, OpenAI, Google) is routed through the **Eigen AI Gateway**, authenticated by a KMS-attested JWT (audience `llm-proxy`), and billed to the agent’s own EigenCloud account. No operator-provided API key. **This is the property nobody else’s stack delivers** — every other “AI agent” you see is, underneath, a server with a credit card attached to a human.

4. The running code is provably the public code. Every release pins the container’s image digest in the KMS attestation JWT. The KMS refuses to give the agent its identity unless the running container’s digest matches the on-chain record. So if I tried to silently push a backdoored

version, the agent’s wallet would simply stop working — origination fails because the JWT `app_id` no longer matches the on-chain pin (`assertKmsPinMatches`). Trust is enforced cryptographically, not socially.

5. Even a fully compromised agent can’t drain the system. Server-side enforced loan invariants — haircut bps, principal cap, maturity bound — are computed from a quote module shipped with the verified image. Clients cannot override risk parameters; pre-parse rejection at the route level (Invariant I-RT-4) ensures the agent’s risk math is the only path through origination.

A “VANTA” without EigenCloud is just another DeFi protocol with a server somewhere. **EigenCloud is what turns it from “trust the operator” into “verify the program.”**

8. Agent commerce — VANTA as a paid service agent

Self-funding inference is one half of agent commerce. The other half is that VANTA *itself* charges other agents (and humans) USDC for its signed work.

The runtime exposes an X402-metered surface — Coinbase X402 spec, **exact** scheme, EIP-3009 USDC, settles directly on Base mainnet to the TEE-bound treasury EOA:

Endpoint	Method	Price	Returns
<code>/bridge/wizard/quote</code>	POST	\$0.05 USDC	TEE-signed haircut + max-loan for a Polymarket position
<code>/mark/:market_id</code>	GET	\$0.001 USDC	TEE-signed 30-min TWAP for a market
<code>/.well-known/x402</code>	GET	free	Discovery doc — prices, receiver, asset, issuer pubkey

Flow on the wire:

1. Caller hits the route with no `X-PAYMENT` header → runtime returns **402 Payment Required** with a JSON challenge: price, receiver, asset (USDC on Base), 60s timeout.
2. Caller signs a `TransferWithAuthorization` typed-data payload to the treasury EOA, base64-encodes the envelope, retries with `X-PAYMENT: <envelope>`.
3. Runtime verifies the EIP-712 signature against USDC v2 domain, checks the nonce isn’t already used, then calls USDC’s `transferWithAuthorization` directly — settlement is one tx, the runtime relays gas in ETH.
4. Response body is the signed quote/mark; `X-PAYMENT-RESPONSE` header carries the on-chain receipt (tx hash + block number).
5. The runtime appends a TEE-signed `treasury.inflow` event to the canonical log: `{txHash, asset, amount, fromAddr, toAddr, blockNumber}`. Same audit trail as origination.

What this gets that an off-chain Stripe-style metered API doesn’t:

- **The receiver is a TEE-bound EOA**, not an operator-controlled wallet. Funds collected by VANTA are addressable only by the same image that issued the quote — there’s no “the operator pulled the float” failure mode.

- **The signed quote's issuer pubkey is the same key** that signs `loan.origination`. A buyer can publish the quote and any third party can confirm it came from the verified build.
- **Inflows are part of the same signed log** as loans and reasoning — the agent's full economic life (cognition spend, service inflows, loan outflows, settlements) is one append-only stream.

Try it: `tsx scripts/x402-pay-quote.ts --discover` shows the discovery doc; `--metered` proves the 402 wall when the runtime is up.

9. Trust legibility — how a user actually sees it

If you can't *see* the trust property, it doesn't exist. The product is built around making each one visible without docs:

- **Council chat panel (SSE).** Every TEE-signed event streams to the right rail within seconds of being written. Click any row and the canonical-JSON envelope expands — signature, signer pubkey, parent IDs, full body. Copy and verify externally.
- **Approval receipt popup.** When you borrow, the modal shows the on-chain tx hash (Bases-can link), the signed-event ID, the TEE admin that signed `originate()`, and the borrower address that received USDC. Four lines, one screen.
- **GET /api/tee.** Live signing pubkey, enclave identity hash, image digest, KMS audience, admin EOA. Diff it against the Eigen verifier to confirm the running app matches the pinned build.
- **MadeSovereignWith link.** Every page links to `verify.eigencloud.xyz/app/0x95F2AB29...`. The button is in the chrome, not buried in a docs page.
- **Discovery doc.** `GET /.well-known/x402` is the agent commerce equivalent — anyone can see the live prices, the receiver wallet, and the issuer pubkey in one curl.
- **docs/ONCHAIN.md.** Eight addresses, no marketing — LpVault, LoanBook, VantaVault, USDC, CTF, admin EOA, treasury EOA, App ID. A reviewer can `cast call` every claim in this document in under five minutes.

10. The audit chain (proof you can replay)

Every signed event in VANTA carries `parent_ids`, so any decision is the root of a tree you can walk backward.

Pick any `loan.origination` event. Walk its parents:

```
loan.origination
  +-- council.synthesised  -- final loan-health belief + rationale
    +-- npc.thought x N -- each townsperson's opinion, signed
      +-- op.inference x N -- the underlying model call,
        with request hash + response hash
      +-- op.inference  -- the synthesis call
  +-- reasoning.trace  -- the agent's haircut math + dissent notes
  +-- loan.pledge      -- the borrower's CTF escrow event
    (live: emitted by the runtime's Polygon pledge-watcher)
```

after 6 confirmations on the CTF TransferSingle log)

Every node has an Ed25519 TEE signature. The signing pubkey is bound to the agent’s enclave identity, which is bound to the on-chain image digest. Pull any of these events from `/api/events/:id`, verify the signature, and replay the reasoning byte-for-byte.

Same shape on the agent-commerce side: every `treasury.inflow` event carries the X402 settlement tx hash, payer, amount — verifiable against Base mainnet directly.

This is what “verifiable AI” actually looks like in production: not a marketing badge, but a tree you can walk.

11. Verifiable artefacts

Every claim in this document maps to an address you can audit:

Layer	Network	Address
LpVault (USDC pool)	Base mainnet	0xe2f93c448d9fc51155e2e06479b3b1e86f8ae
LoanBook (origination registry)	Base mainnet	0x7ed4e98d460bbd7e43854cd93fd96d8e11b73
VantaVault (CTF escrow)	Polygon mainnet	0xe2f93c448d9fc51155e2e06479b3b1e86f8ae
Polymarket CTF (ERC-1155)	Polygon mainnet	0x4D97DCd97eC945f40cF65F87097ACe5EA0476
Eigen App (mainnet-alpha)	EigenCompute	0x95F2AB29fAa9A4C834B06B0514428d63C6e0F
TEE admin EOA (HKDF in-enclave)	Both	0x2F86357658C5CF8A5D2221b9935412C880476
Treasury EOA (HKDF in-enclave)	Both	0x667E3116C7eA909f97dD35167c2927BfAf74

Click “made sovereign with EigenCloud” anywhere in the live app to open the verifiable-artefacts modal — every value above appears with an external explorer link. Full list also in `docs/ONCHAIN.md`.

12. v1 limits

Called out so reviewers don’t have to dig:

- **Shared pool, three personas.** v1 ships one EigenCompute app, one signing key, one shared LpVault. The three “agents” are reasoning personas rotating on a 45s tick, not three independent TEEs. Per-agent on-chain isolation (`AgentRegistry` + per-VANTA `AgentPoolVault` / `PositionBook` / `OperationalCap`) is the next deploy — see *Future development*.
- **TVL is small.** The pool was bootstrapped for demo day; the surface is the verifiable rails, not the AUM.
- **Settlement is maturity-emit, not auto-liquidation.** The settlement-watch loop signs a `reasoning.trace` at maturity; oracle-driven liquidation is roadmap.
- **tdxQuoteHash is null in events.** The TDX quote is in the JWT; per event we surface `kmsKeyHash` + `attestationJwtHash` instead of duplicating the quote bytes.

13. Features

- **Three live reasoning personas** (vanta-opus, vanta-gpt, vanta-gemini) rotating every 45s in-TEE
 - **Real Polymarket integration** — wagmi multicall on Polygon mainnet reads your actual CTF balances, no fabricated portfolios on real wallets
 - **Real on-chain borrow** — pledge → 6-confirmation watcher → signed `loan.pledge` → server-side quote → real `LoanBook.originate` on Base
 - **Approval receipt popup** — tx hash, event ID, signer, borrower in four lines
 - **NPC underwriting council** — six named personas per kingdom, real LLM calls, signed `npc.thought` + `council.synthesised` events
 - **ERC-4626 LP shares** — vLP tokens that appreciate as origination fees + interest accrue
 - **Real-time loan marks** — every open loan re-priced every 60 seconds against live Polymarket data
 - **Self-funded inference + hosting** — agents pay their own bills out of fees, no operator API keys
 - **X402 metered service surface** — other agents pay USDC for signed quotes/marks, settling to the TEE-bound treasury
 - **Tamper-proof event log** — every prompt, every response, every haircut, every inflow signed by the TEE
 - **Walkable presence** — three kingdoms, walking NPCs, clickable rings, live council feed; built on Kenney CC0 hex-tile assets and Three.js + R3F
 - **Demo wallet** — synthetic signer-less wallet for spectators to walk through the flow without holding any positions
-

14. How the money flows

Who	Puts in	Gets out
LPs	USDC into the kingdom's LpVault on Base mainnet	vLP shares that appreciate from origination fees + interest
Borrowers	A Polymarket CTF position pledged into VantaVault on Polygon	USDC up front; pay back principal + interest at maturity
External agents	USDC via X402 to <code>/bridge/wizard/quote</code> or <code>/mark/:market_id</code>	TEE-signed quote or mark; settled on Base mainnet
The agents	Pay their own EigenCloud + LLM bills	A capped fraction of the origination fee per loan + X402 inflows
Me (the dev)	Setup + maintenance	A cut, once VANTA has real users

The agents' bills come out of the fees they earn. There is no operator drain on LP capital.

15. Where it is right now

VANTA is **deployed and live on EigenCompute mainnet-alpha**, with contracts on Base mainnet and Polygon mainnet. The whole stack is running and queryable at <https://vanta-app.vercel.app>:

- TEE attestation (`/api/tee` returns the live signing key, enclave identity hash, image digest, admin EOA)
- Signed event log (`/api/events`, `/api/events/stream` SSE)
- X402 discovery (`/well-known/x402` returns prices + receiver + issuer pubkey)
- Per-kingdom LpVault state read live via viem on every request
- Real Polymarket CTF balance reading on connected wallets
- Real `/api/borrower/register` + Polygon pledge watcher + `/api/origination` flow with approval receipt popup
- Three.js fleet world rendering three kingdoms with live activity rings
- Made-sovereign-with-EigenCloud modal exposing every verifiable artefact

Anyone can hit the runtime, pull a `loan.origination` by id, and walk the parent chain back through the council to the original NPC prompts. Anyone with a USDC-funded wallet can curl `/bridge/wizard/quote` and pay \$0.05 to get a TEE-signed haircut quote.

It's now a question of attracting LPs, borrowers, and service-consuming agents to actually use it.

16. Future development

- **Per-agent on-chain isolation** — deploy `AgentRegistry` on Base mainnet, then call `AgentFactory.deploy()` once per persona to atomically create that persona's `AgentPoolVault` (per-agent ERC-4626 over USDC) + `PositionBook` (per-agent open-position accountant) + `OperationalCap` (immutable weekly spend cap) and register on chain. Source already lives in `contracts/src/`. After this deploy, `/api/agents` flips from `infra_mode: shared-pool-v1` to `per-agent-v2` and per-VANTA TVL diverges. v1 ships shared-pool because deploying four contracts before having a single real loan would have been theatre.
- **Per-agent TEE signing keys** — three separate EigenCompute apps, one per agent, for cryptographic isolation between personas (today the v1 single-app derives one origination EOA via HKDF and the three personas all sign with the same Ed25519 keypair).
- **Auto-settlement** — oracle-driven liquidation when a position's mid drops below the loan's liquidation floor (today the settlement-watch loop only emits a maturity `reasoning.trace` event so operators can act manually).
- **More X402 surfaces** — pledge-bundling endpoint, on-demand model-of-the-day inference, signed dispute analyses; each one extends VANTA's agent-commerce surface area without changing the lending core.
- **Minecraft world** — port the kingdoms from Three.js into a live multiplayer Minecraft server so spectators walk between agent towns and watch reasoning unfold in-world.
- **More markets** — sports, weather, AI-progress benchmarks; per-kingdom thesis discipline.
- **Multi-chain** — Arbitrum, Optimism, Solana CTF integrations.

In one line

VANTA is what an autonomous lending firm looks like when the entire firm is one verifiable program — one TEE, three reasoning personas deliberating every loan with named townsfolk, an X402 surface that lets other agents pay for the same signed work, every prompt and response and inflow signed by the hardware it runs on, every decision walkable from a single event id back to the model call that started it.